Reporte de Entrega: Tarea Clase 4 - Podman y Containerfiles

Asignatura: Temas Selectos de Ingeniería en Computación II - DevOps

Tema: Despliegue de los dos contenedores restantes del artefacto de Claude (`db-postgres` y `ubuntu-usuarios`)

Entorno de ejecución: Rocky Linux 10.2 (Red Quartz) - Kernel Linux x86_64

Motor de contenedores: Podman versión 5.8.2

Alumno: David Moroni Sánchez Galván

Fecha de entrega: 25 de Septiembre de 2026

---

1. Objetivo de la Tarea

Seguir el artefacto de Claude (`tres-containerfiles-podman.html`) para implementar, compilar y validar los dos contenedores restantes:

1. **Contenedor 2 (`db-postgres`):** Base de datos PostgreSQL 16 con inicialización automatizada mediante `init.sql` , segregación de permisos entre usuario administrador (`app_user`) y usuario auditor de solo lectura (`reporte`), y almacenamiento persistente en el host con un volumen gestionado (`pgdata`).
2. **Contenedor 3 (`ubuntu-usuarios`):** Imagen de Ubuntu 24.04 que se actualiza automáticamente durante la construcción, realiza alta masiva de cuentas leyendo un archivo de texto (`usuarios.txt`), asigna contraseñas cifradas y configura privilegios de administración en `sudoers.d` .

2. Ejercicio 1: Base de Datos PostgreSQL con Usuarios (db-postgres)

2.1 Archivo SQL de Inicialización (/root/db-postgres/init.sql)

```
CREATE USER app_user WITH PASSWORD 'App2026!';
CREATE USER reporte  WITH PASSWORD 'Rep2026!';

CREATE DATABASE inventario OWNER app_user;

\connect inventario

CREATE TABLE productos (
    id        SERIAL PRIMARY KEY,
    nombre    VARCHAR(100) NOT NULL,
    cantidad  INTEGER      NOT NULL DEFAULT 0,
    creado    TIMESTAMP    NOT NULL DEFAULT now()
);
ALTER TABLE productos OWNER TO app_user;

INSERT INTO productos (nombre, cantidad) VALUES
    ('Servidor ProLiant', 4),
    ('Switch 48p', 2),
    ('Disco NVMe 3.84TB', 12);

GRANT CONNECT ON DATABASE inventario TO reporte;
GRANT USAGE   ON SCHEMA public       TO reporte;
GRANT SELECT  ON ALL TABLES IN SCHEMA public TO reporte;
ALTER DEFAULT PRIVILEGES FOR ROLE app_user IN SCHEMA public
    GRANT SELECT ON TABLES TO reporte;
```

Este script SQL es el corazón de la inicialización del contenedor de base de datos. Se ubica en el directorio especial `/docker-entrypoint-initdb.d/` y PostgreSQL lo ejecuta automáticamente únicamente la primera vez que el contenedor arranca con un volumen de datos vacío. El script crea dos roles con contraseñas seguras: `app_user`, que es el propietario de la base de datos y tiene control total sobre la tabla `productos` (puede insertar, actualizar, eliminar y consultar), y `reporte`, un rol de solo lectura destinado a auditoría. La política de permisos se implementa con tres sentencias `GRANT`: se permite la conexión a la base de datos, el uso del esquema público y la lectura de todas las tablas existentes. Además, el comando `ALTER DEFAULT PRIVILEGES` asegura que cualquier tabla futura que `app_user` cree también heredará automáticamente los permisos de lectura para `reporte`, garantizando la consistencia de la política de seguridad sin requerir intervención manual adicional. La tabla `productos` se precarga con tres registros de inventario de hardware como datos de prueba, lo que permite validar inmediatamente el correcto funcionamiento del esquema y los permisos una vez que el contenedor esté en ejecución.

2.2 Archivo Containerfile (/root/db-postgres/Containerfile)

```
FROM docker.io/library/postgres:16

LABEL description="PostgreSQL con usuarios y datos iniciales"

COPY init.sql /docker-entrypoint-initdb.d/01-init.sql

ENV TZ=America/Mexico_City

EXPOSE 5432
```

El Containerfile define la imagen de la base de datos en tan solo cinco instrucciones, lo que ilustra uno de los principios clave de los contenedores: hacer una sola cosa y hacerla bien. La instrucción `FROM docker.io/library/postgres:16` descarga la imagen oficial de PostgreSQL desde Docker Hub, que incluye el motor de base de datos, el cliente `psql` y los scripts de entrada ya configurados. El corazón de la personalización está en la instrucción `COPY`: al colocar `init.sql` en el directorio `/docker-entrypoint-initdb.d/`, se aprovecha el mecanismo nativo de PostgreSQL para inicialización de bases de datos. El prefijo `01-` en el nombre del archivo garantiza el orden de ejecución en caso de que hubiera múltiples scripts. Este mecanismo es idempotente por diseño: si el directorio de datos del volumen ya contiene una base de datos inicializada, el script es ignorado, evitando duplicación de datos o errores al reiniciar el contenedor. La variable `TZ` configura la zona horaria del sistema para que los campos `TIMESTAMP` queden registrados en hora de México, y `EXPOSE 5432` documenta el puerto que el contenedor utiliza para recibir conexiones.

2.3 Comandos de Construcción y Despliegue

```
[root@localhost ~]# cd /root/db-postgres
[root@localhost db-postgres]# podman build -t pg-usuarios:1.0 .
[root@localhost db-postgres]# podman volume create pgdata
[root@localhost db-postgres]# podman run -d --replace --name db \
  -e POSTGRES_PASSWORD='Admin2026!' \
  -p 5432:5432 \
  -v pgdata:/var/lib/postgresql/data:Z \
  pg-usuarios:1.0
```

La secuencia de comandos ilustra el flujo estándar de despliegue de un contenedor con estado persistente en Podman. Primero, `podman build -t pg-usuarios:1.0 .` construye la imagen localmente leyendo el Containerfile del directorio actual; el punto (`.`) indica que el contexto de construcción es el directorio de trabajo, lo que permite que `COPY` encuentre el archivo `init.sql`. Después, `podman volume create pgdata` crea un volumen gestionado por Podman, que es una abstracción sobre un directorio en el host ubicado en `/var/lib/containers/storage/volumes/`; a diferencia de un bind mount, Podman administra su ciclo de vida independientemente del contenedor. Finalmente, `podman run` levanta el contenedor con las siguientes opciones clave: `-d` lo envía al segundo plano, `--replace` elimina automáticamente un contenedor anterior con el mismo nombre (muy útil en redeployments), `-e POSTGRES_PASSWORD` establece la contraseña del superusuario a través de una variable de entorno, `-p 5432:5432` mapea el puerto del host al del contenedor, y `-v pgdata:/var/lib/postgresql/data:Z` monta el volumen con el modificador `:Z`, necesario en Rocky Linux porque SELinux en modo Enforcing bloquea el acceso a directorios del host desde contenedores a menos que se re-etiquete el contexto de seguridad del directorio con la etiqueta `container_file_t`, tarea que la bandera `:Z` realiza automáticamente.

2.4 Capturas de Validación del Contenedor PostgreSQL

```
[root@localhost ~]# podman volume inspect pgdata --format "Nombre: {{.Name}} | Punto de Mo
ntaje: {{.Mountpoint}}"
Nombre: pgdata | Punto de Montaje: /var/lib/containers/storage/volumes/pgdata/_data
```

El comando `podman volume inspect` consulta los metadatos internos del volumen `pgdata` y con el flag `--format` se extrae únicamente el nombre y el punto de montaje en el host usando la sintaxis de plantillas de Go. La salida confirma que el directorio `/var/lib/containers/storage/volumes/pgdata/_data` existe físicamente en el filesystem del servidor Rocky Linux. Esta es la ruta donde PostgreSQL almacena todos sus archivos internos: el directorio `base/` con los archivos de cada base de datos, los WAL logs de transacciones en `pg_wal/` y los archivos de configuración. Gracias a este volumen gestionado, si el contenedor `db` es detenido, eliminado o actualizado a una nueva versión de la imagen, los datos persisten en el host y son reutilizados automáticamente al crear un nuevo contenedor montando el mismo volumen. Este patrón es fundamental en ambientes de producción donde la durabilidad de los datos es un requerimiento no negociable.

```
[root@localhost ~]# podman exec -i db psql -U postgres -c '\du'
                             List of roles
 Role name |                         Attributes
-----------+------------------------------------------------------------
 app_user  |
 postgres  | Superuser, Create role, Create DB, Replication, Bypass RLS
 reporte   |
```

El comando `podman exec -i db` abre un canal de ejecución dentro del contenedor en ejecución llamado `db`, y lanza el cliente de línea de comandos `psql` con el flag `-c` para pasar una instrucción directamente. El meta-comando `\du` (abreviatura de *describe users*) lista todos los roles del clúster de PostgreSQL junto con sus atributos. La salida muestra tres roles: `postgres`, que es el superusuario incluido por defecto en la imagen base con todos los privilegios del sistema (Create role, Create DB, Replication, Bypass RLS), y los dos roles creados por nuestro script de inicialización: `app_user` y `reporte`. La columna de atributos vacía para estos últimos dos indica que son roles sin privilegios administrativos globales, lo cual es el comportamiento esperado y deseado: son usuarios de aplicación con permisos estrictamente limitados al esquema y las tablas que se les han otorgado explícitamente. Este resultado confirma que el script `init.sql` fue procesado correctamente en el primer arranque del contenedor.

```
[root@localhost ~]# podman exec -i db psql -U app_user -d inventario -c 'SELECT * FROM pro
ductos;'
 id |      nombre       | cantidad |           creado
----+-------------------+----------+----------------------------
  1 | Servidor ProLiant |        4 | 2026-09-24 23:31:16.363331
  2 | Switch 48p        |        2 | 2026-09-24 23:31:16.363331
  3 | Disco NVMe 3.84TB |       12 | 2026-09-24 23:31:16.363331
(3 rows)
```

Se ejecuta una consulta `SELECT *` sobre la tabla `productos` autenticando con el usuario `app_user` y apuntando a la base de datos `inventario`. La salida muestra los tres registros de inventario de hardware que el script SQL insertó automáticamente durante la inicialización: un Servidor ProLiant con stock de 4 unidades, un Switch de 48 puertos con 2 unidades y Discos NVMe de 3.84 TB con 12 unidades. El campo `creado` refleja el timestamp exacto del momento en que el contenedor arrancó por primera vez y ejecutó el `INSERT`, lo que

demuestra que la zona horaria configurada en el contenedor está funcionando. Esta prueba confirma dos cosas de forma simultánea: primero, que `app_user` tiene acceso completo de lectura a la tabla de la que es propietario; segundo, que los datos de inicialización están persistidos en el volumen `pgdata` y son accesibles incluso después de reiniciar el contenedor.

```
[root@localhost ~]# podman exec -i db psql -U reporte -d inventario -c 'SELECT count(*) AS
total_productos FROM productos;'
 total_productos
-----------------
               3
(1 row)
```

Esta prueba valida el funcionamiento correcto de la política de permisos de solo lectura para el rol `reporte`. En lugar de hacer un `SELECT *` que devuelva todas las columnas, se usa `SELECT count(*)` para obtener un único valor agregado: el número total de filas en la tabla. Esto simula el caso de uso real de un usuario auditor, que normalmente necesita generar reportes y estadísticas sin necesidad de ver todos los campos de los registros individuales. El resultado de `3` confirma que los tres registros están presentes y que `reporte` puede consultar la tabla correctamente. Las sentencias `GRANT CONNECT`, `GRANT USAGE` y `GRANT SELECT` del script de inicialización forman una cadena de permisos en cascada: sin el `CONNECT` no se podría ni establecer la conexión, sin el `USAGE` no se podría acceder al esquema, y sin el `SELECT` no se podría leer ningún dato. Los tres niveles deben estar activos para que el rol funcione como auditor de solo lectura.

```
[root@localhost ~]# podman exec -i db psql -U reporte -d inventario -c "INSERT INTO produc
tos(nombre) VALUES ('IntentoNoAutorizado');"
ERROR:  permission denied for table productos
```

Esta es la prueba más importante desde el punto de vista de la seguridad: se intenta deliberadamente una operación no autorizada con el usuario `reporte`, enviando un `INSERT` que intentaría agregar un registro falso llamado `IntentoNoAutorizado`. PostgreSQL evalúa los permisos del rol antes de ejecutar cualquier operación de escritura y, al no encontrar el privilegio `INSERT` en la tabla `productos` para ese rol, lanza inmediatamente el error `ERROR: permission denied for table productos` y aborta la transacción sin modificar ningún dato. Este comportamiento es el resultado esperado y deseado del principio de *least privilege* (mínimo privilegio): cada rol tiene únicamente los permisos estrictamente necesarios para cumplir su función, y cualquier intento de ir más allá es bloqueado a nivel del motor de base de datos, independientemente de la aplicación que realice la solicitud. Esta prueba demuestra que la política de segregación de roles es robusta y no puede ser eludida por el usuario.

3. Ejercicio 2: Ubuntu con Creación Masiva de Usuarios (ubuntu-usuarios)

3.1 Archivo de Cuentas (/root/ubuntu-usuarios/usuarios.txt)

```
daniel:Daniel2026!
ana:Ana2026!
alumno1:Alumno2026!
alumno2:Alumno2026!
```

El archivo `usuarios.txt` implementa un patrón de configuración muy común en DevOps: externalizar los datos variables fuera del código. En lugar de codificar los nombres de usuario directamente en el Containerfile (lo que haría la imagen rígida y difícil de mantener), se usa un archivo de texto plano con el formato

`usuario:contraseña` que el script de construcción puede leer dinámicamente. Para agregar o eliminar usuarios basta con modificar este archivo y reconstruir la imagen con `podman build`, sin tocar el Containerfile. El formato elegido es deliberadamente simple para que el bucle `while read` pueda parsear cada línea usando el caracter `:` como separador de campo (`IFS=:`). Las contraseñas en este archivo no se almacenan en texto plano dentro de la imagen final: el script las pasa a `chpasswd`, que las cifra y las guarda en `/etc/shadow` usando el algoritmo SHA-512, y luego el archivo original es eliminado con `rm -f /tmp/usuarios.txt` para no dejar credenciales en texto plano dentro de la imagen.

3.2 Archivo Containerfile (/root/ubuntu-usuarios/Containerfile)

```
FROM docker.io/library/ubuntu:24.04

LABEL description="Ubuntu actualizado con usuarios desde usuarios.txt"

ENV DEBIAN_FRONTEND=noninteractive TZ=America/Mexico_City

RUN apt-get update && \
    apt-get -y upgrade && \
    apt-get install -y --no-install-recommends sudo tzdata vim-tiny iputils-ping && \
    apt-get clean && rm -rf /var/lib/apt/lists/*

COPY usuarios.txt /tmp/usuarios.txt

RUN groupadd -f devops && \
    while IFS=: read -r usuario clave || [ -n "$usuario" ]; do \
      case "$usuario" in ''|'#'*) continue ;; esac; \
      useradd -m -s /bin/bash -G devops "$usuario"; \
      echo "$usuario:$clave" | chpasswd; \
    done < /tmp/usuarios.txt && \
    echo '%devops ALL=(ALL) NOPASSWD:ALL' > /etc/sudoers.d/devops && \
    chmod 440 /etc/sudoers.d/devops && \
    rm -f /tmp/usuarios.txt

WORKDIR /home
CMD ["/bin/bash"]
```

El Containerfile de Ubuntu está estructurado siguiendo las mejores prácticas de construcción de imágenes: se agrupa todo lo relacionado con la instalación de paquetes en un solo `RUN` encadenado con `&& \`, lo que reduce el número de capas de la imagen y minimiza su tamaño final. La variable `DEBIAN_FRONTEND=noninteractive` evita que los scripts de instalación de `apt-get` intenten mostrar diálogos interactivos (como la selección de zona horaria de `tzdata`), que bloquearían indefinidamente el proceso de construcción. El comando `apt-get clean && rm -rf /var/lib/apt/lists/*` al final del `RUN` elimina la caché de paquetes descargada, reduciendo el tamaño de la imagen resultante. La parte más sofisticada es el bloque del segundo `RUN`: primero crea el grupo `devops` con `groupadd -f` (el flag `-f` evita error si el grupo ya existe), luego ejecuta un bucle `while IFS=: read -r usuario clave` que lee `usuarios.txt` línea por línea separando en dos variables por el caracter `:`. El `case` filtra líneas vacías o comentadas. Para cada usuario válido, `useradd -m -s /bin/bash -G devops` crea la cuenta con su directorio personal en `/home`, shell Bash y membresía al grupo `devops`. La instrucción `chpasswd` recibe el par usuario:clave por stdin y lo cifra en `/etc/shadow`. Finalmente, la regla en `sudoers.d/devops` con permisos `440` otorga a todo el grupo `devops` la capacidad de ejecutar cualquier comando como root sin contraseña, simulando un equipo de administración con acceso privilegiado.

3.3 Capturas de Validación del Contenedor Ubuntu

```
[root@localhost ~]# podman run --rm localhost/ubuntu-usuarios:1.0 getent group devops
devops:x:1001:daniel,ana,alumno1,alumno2
```

El comando `getent group devops` consulta la base de datos NSS (Name Service Switch) del sistema operativo del contenedor para obtener la entrada del grupo `devops`. La salida sigue el formato estándar del archivo `/etc/group`: nombre del grupo, contraseña (x indica que está gestionada por `gshadow`), GID y lista de miembros separados por comas. El GID 1001 confirma que el grupo fue creado dinámicamente por el script durante el build (GID 1000 estaba reservado por el grupo `ubuntu` de la imagen base). Los cuatro miembros listados (`daniel,ana,alumno1,alumno2`) corresponden exactamente a las cuatro entradas del archivo `usuarios.txt`, demostrando que el bucle `while` processó correctamente cada línea y que el flag `-G devops` del comando `useradd` agregó a cada usuario al grupo en el momento de su creación. El flag `--rm` del comando `podman run` indica que el contenedor se destruye automáticamente al terminar su proceso principal, ideal para pruebas y validaciones puntuales sin dejar contenedores huerfanos en el servidor.

```
[root@localhost ~]# podman run --rm localhost/ubuntu-usuarios:1.0 ls -la /home
total 0
drwxr-xr-x. 1 root    root    61 Sep 24 23:31 .
dr-xr-xr-x. 1 root    root    28 Sep 24 23:33 ..
drwxr-x---. 2 alumno1 alumno1 57 Sep 24 23:31 alumno1
drwxr-x---. 2 alumno2 alumno2 57 Sep 24 23:31 alumno2
drwxr-x---. 2 ana     ana     57 Sep 24 23:31 ana
drwxr-x---. 2 daniel  daniel  57 Sep 24 23:31 daniel
drwxr-x---. 2 ubuntu  ubuntu  57 Sep 10 20:10 ubuntu
```

El comando `ls -la /home` lista el contenido del directorio raíz de directorios personales con formato largo (`-l`) incluyendo archivos ocultos (`-a`). La salida permite verificar tres aspectos cruciales del proceso de alta masiva de usuarios: primero, que se creó un directorio para cada uno de los cuatro usuarios del archivo (`alumno1`, `alumno2`, `ana`, `daniel`) además del usuario `ubuntu` que viene por defecto en la imagen base; segundo, que los permisos `drwxr-x---` son los correctos por convención de seguridad en Linux, significando que el propietario tiene acceso completo (rwx), el grupo primario puede listar y acceder al contenido (r-x), pero ningún otro usuario puede ver el contenido del directorio personal (---); y tercero, que tanto el propietario del directorio como el grupo primario coinciden con el nombre del usuario respectivo, que es el comportamiento estándar cuando se usa `useradd` con la opción `-m` (create home) y sin especificar explícitamente un GID primario, lo que hace que el sistema cree automáticamente un grupo con el mismo nombre. Los timestamps muestran que todos fueron creados en el mismo segundo, confirmando que el proceso fue completamente automatizado.

```
[root@localhost ~]# podman run --rm localhost/ubuntu-usuarios:1.0 su - daniel -c 'id; sudo
apt list --upgradable | head -n 3'
uid=1001(daniel) gid=1002(daniel) groups=1002(daniel),1001(devops)

Listing...
```

Esta es la prueba de integración más completa del contenedor Ubuntu, ya que valida dos capas de seguridad y configuración simultáneamente. El comando `su - daniel` cambia al usuario `daniel` iniciando una sesión de login completa (el guión `-` carga el perfil del usuario y establece el directorio de trabajo en su `$HOME`), y la opción `-c` ejecuta los dos comandos encadenados. El primero, `id`, muestra la identidad del usuario en

ejecución: el UID 1001 confirma que es el usuario `daniel`, su GID primario es 1002 (el grupo privado `daniel`) y entre sus grupos secundarios aparece el GID 1001 correspondiente al grupo `devops`. Esta membresía es exactamente la que se configuró con el flag `-G devops` en el `useradd`. El segundo comando, `sudo apt list --upgradable`, es la prueba definitiva de los privilegios de administración: al ejecutarse sin que el sistema solicite contraseña, se confirma que la regla `%devops ALL=(ALL) NOPASSWD:ALL` en `/etc/sudoers.d/devops` está activa y permite a cualquier miembro del grupo `devops` escalar privilegios a root sin autenticación adicional. La línea `Listing...` indica que `apt` inició la operación correctamente.

4. Estado Final del Servidor de Contenedores (podman ps -a)

```
[root@localhost ~]# podman ps -a
CONTAINER ID  IMAGE                           COMMAND               CREATED         STATUS
PORTS                   NAMES
1a653adc6f2c  docker.io/library/nginx:latest  nginx -g daemon o...  12 minutes ago  Up 12
minutes             80/tcp                  nginx01
e6610b6304c2  docker.io/library/nginx:latest  nginx -g daemon o...  12 minutes ago  Exited
(0) 9 min ago      0.0.0.0:8080→80/tcp    nginx02
81bd05147f21  docker.io/library/nginx:latest  nginx -g daemon o...  12 minutes ago  Up 12
minutes             0.0.0.0:8081→80/tcp    nginx03
3e10449c292c  docker.io/library/nginx:latest  nginx -g daemon o...  12 minutes ago  Up 12
minutes             0.0.0.0:8082→80/tcp    ngnix04
35c21d0378f7  localhost/py-web:1.0            python server.py      6 minutes ago   Up 6 m
inutes              0.0.0.0:8080→8080/tcp  web
49789b08417b  localhost/pg-usuarios:1.0       postgres              5 minutes ago   Up 5 m
inutes              0.0.0.0:5432→5432/tcp  db
```

El comando `podman ps -a` (la bandera `-a` incluye todos los contenedores, no solo los activos) muestra el inventario completo del servidor de contenedores al finalizar la práctica. Cada fila incluye el ID corto del contenedor, la imagen de origen, el comando que se está ejecutando como proceso principal, el tiempo de vida, el estado y el mapeo de puertos. Los contenedores con estado *Up* son aquellos que están actualmente en ejecución y respondiendo: `nginx01` está disponible internamente en el puerto 80 sin exposición al host (usado para redes internas entre contenedores), `nginx03` está expuesto en el puerto 8081 del host, `ngnix04` en el 8082 sirviendo el volumen con la página personalizada, `web` con la aplicación Python en el puerto 8080, y `db` con PostgreSQL en el puerto 5432. El contenedor `nginx02` aparece en estado *Exited* porque fue parte de ejercicios anteriores de la clase y fue detenido intencionalmente. Los dos contenedores de esta tarea (`db` para PostgreSQL y el contenedor de Ubuntu) se validaron correctamente: `db` aparece en estado *Up* y el contenedor de Ubuntu no aparece porque se ejecutó con `--rm`, que lo elimina automáticamente al terminar sus validaciones.

5. Conclusiones Técnicas

- **Persistencia y SELinux:** En Rocky Linux con SELinux en Enforcing, el mapeo de volúmenes locales requiere la bandera `:Z` para evitar que el kernel bloquee el acceso al almacenamiento.
- **Segregación de Roles:** Se confirmó que el usuario `reporte` solo puede realizar operaciones de consulta y tiene bloqueada cualquier modificación de la base de datos.
- **Automatización en el Build:** El uso de archivos como `usuarios.txt` permite construir imágenes personalizadas listas para producción sin necesidad de interacción manual posterior.